

Local AI Agent Architecture for Rust on CPU-Only Machine with Editor Integration

System Info

- CPU: Intel Core i7-10710U (6 cores / 12 threads @ 1.1GHz, turbo 4.7GHz)
- RAM: 15 GB
- No GPU (CPU-only inference)
- OS: Ubuntu 64-bit

Goal

Create a modular local AI agent in Rust with:

- Terminal-based chatbot (TUI)
- Optional WebSocket server for browser UI
- PDF reading & summarization
- Linux shell command execution with restrictions
- Short-term chat history (with optional :save)

Model Requirements

- Use a quantized GGUF model ($\leq 7B$) like Mistral 7B Q4KM
- Compatible with llama.cpp
- Smooth CPU-only performance (no GPU needed)
- Example download command for model

Architecture

```
/rust-ai-agent/ |— src/ | |— main.rs | |— chat_ui.rs | |— llama_interface.rs | |— pdf_reader.rs | |— shell_runner.rs | |— ws_server.rs |— models/ |— model.gguf |— Dockerfile |— Cargo.toml
```

Security

- Only run shell commands explicitly prefixed (:run ...)
- Block dangerous commands (rm, shutdown, etc.)
- Sanitize all inputs and outputs
- WebSocket binds only to localhost
- Docker container to sandbox the app



Optimization

- Use Q4KM / Q5_1 quantized models

- Limit LLM context to 1024-2048 tokens
- Chunk large PDFs
- Use threaded subprocesses for smooth handling
- Efficient memory use in Rust

Deployment (Docker)

- Dockerfile installs Rust and llama.cpp
- Mount or copy models and PDFs
- Agent runs CLI chatbot and/or WebSocket server in container

Editor Integration

Neovim:

- Lua plugin to:
 - Send selected text to the Rust agent (CLI or WebSocket)
 - Display response in floating window
 - Keymap <leader>ai for prompt

VSCode:

- Simple extension connects to ws://localhost:3030/chat
- Selection-to-AI → inline or sidebar response
- Can reuse terminal or input popup

Chat Commands

- `:load myfile.pdf` — load PDF content
- `:run <cmd>` — run safe shell command
- `:save` — save chat history
- ASCII-style terminal interface